

SCJP 1.6 (CX-310-065 , CX-310-066)

Subject: Flow Control,Exceptions, and Assertions

Total Questions : 26

Prepared by : <http://www.javacertifications.net>

SCJP 6.0: Flow Control, Exception and Assertions

Questions Question - 1

What is the output for the below code?

```
public class Test {  
  
    public static void main(String... args) {  
        int arr[] = {2,4,8,16,32};  
        int result = 0;  
        for (int i=0; i < arr.length; i++){  
            result = result + arr[i];  
        }  
        System.out.println("Output old: " + result);  
        result = 0;  
        for(int a:arr)  
        {  
            result = result + a;  
        }  
        System.out.println("Output new: "+ result);  
  
    }  
}
```

- 1.Output old: 62 Output new: 62
- 2.Output old: 62 Output new: 0
- 3.Compile error
- 4.None of the above

Explanation :

A is the correct answer.

for(int a:arr) is "for each int a in arr." in jdk 1.5

Question - 2

What is the output for the below code ?

```
public class Test {  
  
    public static void main(String... args) {  
        Vector vec = new Vector();  
  
        vec.addElement(2);  
    }  
}
```

```

        vec.addElement(4);

        for(Object obj:vec)
            System.out.println("Output: "+(Integer)obj);
    }
}

```

- 1.Compile error because can't add primitive type in vector.
- 2.for(Object obj:vec) not correct syntax
- 3.Output: 2 Output: 4
- 4.None of the above

Explanation :

C is the correct answer.

JDK 1.5, 1.6 allows add primitive type in vector. and for(Object obj:vec) is correct syntax

Question - 3

Which statement is true about for each for the bellow code?

```

void cancelAll(Collection<TimerTask> c) {
    for (TimerTask t : c)
        t.cancel();
}

```

- 1.The for-each loop hides the iterator, when you so you cannot call remove
- 2.it is not usable for loops where you need to replace elements in a list
- 3.Both are true
- 4.None of the above

Explanation :

C is the correct answer.

The for-each loop hides the iterator, so you cannot call remove. Therefore, the for-each loop is not usable for filtering. Similarly it is not usable for loops where you need to replace elements in a list or array as you traverse it. Finally, it is not usable for loops that must iterate over multiple collections in parallel

Question - 4

What is the output ?

```

public class Test {

    public static void main(String... args) {
        ArrayList<Integer> list = new ArrayList<Integer>();
        list.add(1);
        list.add(2);
    }
}

```

```

        list.add(3);

        for(int i:list)
            System.out.println(i);

    }
}

```

1.1 2 3

2. Compile error , can't add primitive type in ArrayList
3. Compile error on for(int i:list) , Incorrect Syntax
4. None of the above

Explanation :

A is the correct answer.

JDK 1.5, 1.6 allows add primitive type in ArrayList and for(int i:list) is also correct.
for(int i:list) is same as for(int i=0; i

Question - 5

enhanced for loop can't be used at _____ ?

1. To remove elements as you traverse collections
2. To modify the current slot in an array or list
3. To iterate over multiple collections or arrays
4. All of the above

Explanation :

D is the correct answer.

It's important to note that the enhanced for loop can't be used everywhere. You can't use the enhanced for loop: To remove elements as you traverse collections To modify the current slot in an array or list To iterate over multiple collections or arrays

Question - 6

Which statement is true about Error?

1. An Error is a subclass of Throwable
2. An Error is a subclass of Exception
3. Error can not catch using catch blok.
4. None of the above

Explanation :

A is the correct answer.

An Error is a subclass of Throwable that indicates serious problems that a reasonable application should not try to catch.

Question - 7

NullPointerException thrown when

1. Accessing or modifying the field of a null object
2. Taking the length of null as if it were an array
3. Calling the instance method of a null object
4. None of the above

Explanation :

A, B and C is the correct answer.

java.lang.Exception extends java.lang.Throwable.

Question - 8

Is this statement is true?

"java.lang.Throwable class is superclass of all Exceptions"

1. true
2. false
3. can't say
4. None of the above

Explanation :

A is the correct answer.

java.lang.Exception extends java.lang.Throwable.

Question - 9

Is the bellow statement is true?

"ArrayIndexOutOfBoundsException is a type of RuntimeException"

1. true
2. false
3. None of the above
4. None of the above

Explanation :

A is the correct answer.

java.lang.Object java.lang.Throwable java.lang.Exception
java.lang.RuntimeException java.lang.IndexOutOfBoundsException
java.lang.ArrayIndexOutOfBoundsException

Question - 10

What is the output ?

```
public class Test {  
    public static void main(String... args) {  
  
        String strarr[] = new String[2] ;  
        System.out.println(strarr[2]);  
  
    }  
  
}
```

- 1.Compile with error
- 2.java.lang.ArrayIndexOutOfBoundsException
- 3.None of the above
- 4.None of the above

Explanation :

B is the correct answer.

String strarr[] = new String[2] has size 2 , and strarr[2] is trying to access third element. array starts with strarr[0]. So ArrayIndexOutOfBoundsException.

Question - 11

What is the output ?

```
public class Test {  
    public static void main(String... args) {  
  
        Object x = new Integer(0) ;  
        System.out.println((String)x) ;  
  
    }  
  
}
```

- 1.Compile with error
- 2.java.lang.ClassCastException
- 3.None of the above
- 4.None of the above

Explanation :
B is the correct answer.

trying to cast integer to String.

Question - 12

What will be the result of compiling the following code:

```
public class Test {  
    public static void main(String... args) {  
        Test t = new Test();  
        System.out.println(t.getName());  
    }  
  
    public String getName() throws Exception  
    {  
        return "das";  
    }  
}
```

- 1.the code will compile and print das
- 2.the code will compile an print null
- 3.Complile time error.
- 4.None of the above

Explanation :
C is the correct answer.

Unhandled exception type Exception. Throws exception should catch by caller.

Question - 13

What will be the result of compiling the following code:

```
public class SuperClass {  
    public int doIt(String str, Integer... data) throws Exception{  
        String signature = "(String, Integer[])";  
        System.out.println(str + " " + signature);  
        return 1;  
    }  
}  
  
public class SubClass extends SuperClass{  
  
    public int doIt(String str, Integer... data)  
    {  
        String signature = "(String, Integer[])";  
        System.out.println("Overridden: " + str + " " + signature);  
    }  
}
```

```

        return 0;
    }

    public static void main(String... args)
    {
        SuperClass sb = new SubClass();
        sb.doIt("hello", 3);
    }
}

```

1. Overridden: hello (String, Integer[])
2. hello (String, Integer[])
3. Compile time error.
4. None of the above

Explanation :

C is the correct answer.

Unhandled exception type Exception.

Question - 14

What will be the result of compiling the following code:

```

public class SuperClass {
    public int doIt(String str, Integer... data) throws
    ArrayIndexOutOfBoundsException{
        String signature = "(String, Integer[])";
        System.out.println(str + " " + signature);
        return 1;
    }
}

public class SubClass extends SuperClass{

    public int doIt(String str, Integer... data) throws Exception
    {
        String signature = "(String, Integer[])";
        System.out.println("Overridden: " + str + " " + signature);
        return 0;
    }

    public static void main(String... args)
    {
        SuperClass sb = new SubClass();
        try{
            sb.doIt("hello", 3);
        }catch(Exception e){

        }

    }
}

```

}

1. Overridden: hello (String, Integer[])
2. hello (String, Integer[])
3. The code throws an Exception at Runtime.
4. Compile with error

Explanation :

D is the correct answer.

Exception Exception is not compatible with throws clause in SuperClass.doIt(String, Integer[]). The same exception or subclass of that exception is allowed.

Question - 15

When the application Thrown NullPointerException ?

1. Calling the instance method of a null object
2. Accessing or modifying the field of a null object
3. Taking the length of null as if it were an array
4. All of the above

Explanation :

D is the correct answer.

NullPointerException Thrown when an application attempts to use null in a case where an object is required. These include: Calling the instance method of a null object. Accessing or modifying the field of a null object. Taking the length of null as if it were an array. Accessing or modifying the slots of null as if it were an array. Throwing null as if it were a Throwable value. Applications should throw instances of this class to indicate other illegal uses of the null object.

Question - 16

What will be the result of compiling the following code:

```
public class Test {  
    public static void main(String... args) {  
        Integer.parseInt("hello");  
    }  
}
```

1. The code does not compile.
2. 0
3. The code throws an NullPointerException at Runtime.
4. java.lang.NumberFormatException at runtime.

Explanation :

B is the correct answer.

NumberFormatException Thrown to indicate that the application has attempted to convert a string to one of the numeric types, but that the string does not have the appropriate format

Question - 17

_____ Thrown if the Java Virtual Machine or a ClassLoader instance tries to load in the definition of a class ?

- 1.NoClassDefFoundError
- 2.NoClassDefFoundException
- 3.None of the above
- 4.None of the above

Explanation :

B is the correct answer.

NoClassDefFoundError Thrown if the Java Virtual Machine or a ClassLoader instance tries to load in the definition of a class (as part of a normal method call or as part of creating a new instance using the new expression) and no definition of the class could be found.

Question - 18

What is the result of executing the following code, using the parameters 3 and 0:

```
public void divide(int a, int b) {  
    try {  
        int c = a / b;  
    } catch (Exception e) {  
        System.out.print("Exception ");  
    } finally {  
        System.out.println("Finally");  
    }  
}
```

- 1.Prints out: Exception Finally
- 2.Prints out: Finally
- 3.Prints out: Exception
- 4.None of the above.

Explanation :

A is the correct answer.

finally block always executed whether exception occurs or not. 3/0 throw exception.

Question - 19

What is the result of executing the following code, using the parameters 0 and 3 ?

```
public void divide(int a, int b) {  
    try {  
        int c = a / b;  
    } catch (Exception e) {  
        System.out.print("Exception ");  
    } finally {  
        System.out.println("Finally");  
    }  
}
```

1. Prints out: Exception Finally
2. Prints out: Finally
3. Prints out: Exception
4. None of the above.

Explanation :

B is the correct answer.

finally block always executed whether exception occurs or not. $0/3 = 0$

Question - 20

```
try{  
    File f = new File("a.txt");  
}catch(Exception e){  
  
}catch(IOException io){  
  
}
```

Is this code create new file name a.txt ?

1. True
2. False
3. Compilation Error
4. None

Explanation :

C is the correct answer.

IOException is unreachable to compiler because all exception is going to catch by Exception block.

Question - 21

What happens when the following code is compiled and run.
Select the one correct answer.

```
for(int i = 2; i < 4; i++)  
    for(int j = 2; j < 4; j++)  
        if(i < j)  
            assert i!=j : i;
```

- 1.The class compiles and runs, but does not print anything.
- 2.The number 2 gets printed with AssertionError
- 3.compile error
- 4.The number 3 gets printed with AssertionError

Explanation :

A is the correct answer.

When if condition returns true, the assert statement also returns true. Hence
AssertionError does not get generated. .

Question - 22

What happens when the following code is compiled and run.
Select the one correct answer.

```
for(int i = 2; i < 4; i++)  
    for(int j = 2; j < 4; j++)  
        assert i!=j : i;
```

- 1.The class compiles and runs, but does not print anything.
- 2.The number 2 gets printed with AssertionError
- 3.compile error
- 4.The number 3 gets printed with AssertionError

Explanation :

B is the correct answer.

When i and j are both 2, assert condition is false, and AssertionError gets generated. .